Hackathon Squad

Abstract

This report details the algorithmic progression in solving the Maximum Weight Independent Set (MWIS) problem on large-scale graphs ($N \leq 200,000$) under a strict 295-second time limit. We compare two distinct meta-heuristic engines: the initial Iterated Local Search (ILS) approach and the advanced Adaptive Biased Forest Randomized (ABFR) engine. The transition from blind heuristic approximation to exact subproblem reduction via Tree Dynamic Programming (DP) yielded a significant performance breakthrough, crossing the 100% global relative yield threshold across the benchmark dataset.

1 Introduction

The Maximum Weight Independent Set (MWIS) problem is a classic NP-Hard optimization challenge. The objective is to find a set of mutually non-adjacent vertices in a weighted graph such that the sum of their weights is maximized. Given the scale of the graphs (up to 200,000 vertices and edges) and the rigid computation time limit (295 seconds), exact global solvers are computationally infeasible. Consequently, finding highly optimal solutions requires advanced meta-heuristics and fast local evaluations.

2 Phase 1: Iterated Local Search (ILS)

The initial architecture relied on an Iterated Local Search (ILS) framework, a standard approach in competitive programming for constraint satisfaction problems.

2.1 Algorithm Pipeline

- **Greedy Initialization:** Nodes were sorted using the heuristic ratio $\frac{S[i]}{deg(i)+1}$ and inserted into the independent set if no conflicts existed.
- **Local Search (1-swaps & 2-swaps):** The algorithm utilized an $O(1)$ conflict array (XOR-swap queue) to rapidly identify if swapping one node out and a better node in would improve the score.
- **Random Destruction:** To escape local optima, 10% of the current team was randomly dropped when the local search converged, forcing the algorithm to rebuild from a damaged state.

2.2 Limitations

While heavily optimized, the ILS engine suffered from the “Local Optima Trap." In dense graphs or massive sparse structures, 1-swaps and 2-swaps are insufficient to escape deep performance craters. The algorithm spent the majority of its 5-minute window blindly guessing and patching, leading to suboptimal yields on complex test cases.

3 Phase 2: Adaptive Biased Forest Randomized (ABFR) Engine

To break the limitations of ILS, the architecture was completely overhauled. The ABFR engine adapts the academic FR14 approximation algorithm into a high-performance execution loop, shifting the paradigm from *heuristic approximation* to *exact subproblem reduction*.

3.1 The Mathematical Loophole

While MWIS is NP-Hard on general graphs, it is solvable in exact linear time $O(V)$ on a **Forest** (a graph with no cycles) using Dynamic Programming. The ABFR engine leverages this structural property.

3.2 Algorithm Pipeline

1. **Heuristic Permutation:** Nodes are assigned a noisy score $\beta_i = \left(\frac{S[i]}{deg(i)+1}\right) \times \text{noise}(0.8, 1.2)$. Intermittently, deterministic forward/backward sweeps are applied to prevent chaos on uniform-weight subgraphs.
2. **FR14 Forest Filter:** Iterating through the sorted nodes, a vertex u is added to a set F only if it connects to ≤ 1 node already in F. This mathematically guarantees that the induced graph on F is a cycle-free forest.
3. **Exact Tree DP:** A post-order traversal DP calculates the exact optimal MWIS for the extracted forest. Two states are maintained for each node u:

$$DP[u][0] = \sum_{v \in \text{children}} \max(DP[v][0], DP[v][1])$$
$$DP[u][1] = S[u] + \sum_{v \in \text{children}} DP[v][0]$$

4. **Greedy Patch:** A final $O(1)$ sweep sweeps up any remaining vertices in the original graph that do not conflict with the optimal forest team.

4 Comparative Results & Empirical Analysis

The benchmark consisted of 23 test cases with varying topologies (dense, sparse, grid, cycle, star, and massive $N = 200,000$ graphs).

Table 1: Performance Comparison: ILS vs. ABFR (23 Test Cases)

Metric	ILS Engine	ABFR Engine	Improvement
Accepted (Exact Match)	15	**15**	-
Above Target (New Best)	3	**6**	**+100%**
Below Target (Suboptimal)	5	**2**	**-60%**
Overall Success Rate	78.2%	**91.3%**	**+13.1%**
Global Relative Yield	99.99%	**100.62%**	**Crossed 100%**

4.1 Topological Breakthroughs & Bottlenecks

- **Dense Graphs (e.g., input_15, +8.80%):** Dense graphs paralyze local search heuristics. The FR14 filter effortlessly bypassed these dense clusters, allowing the exact DP to pull nearly 9% more weight.
- **Cycle Graphs (e.g., input_05, +3.25%):** By randomly breaking a single edge to form a forest, the Tree DP instantly calculated the mathematically perfect independent set for the rest of the ring.
- **Massive Sparse Graphs (e.g., input_18, +0.07%):** ABFR successfully chewed through 200,000 nodes and edges to set a new global high score, overcoming previous ILS timeouts.
- **Uniform Graph Stabilization (e.g., input_13):** Flat topologies with identical skill levels previously caused random tie-breakers to shatter the forest structure. Replacing pure noise with deterministic, sequential sweeps guaranteed unbroken paths, allowing the Tree DP to extract 100% optimal scores.
- **Bottleneck – Time Constraints (input_16 & 17, -0.06% to -1.16%):** For massive sparse graphs (100,000+ nodes), the 295-second limit simply isn't long enough for the engine to randomly generate the specific forest configuration needed to unlock the final few nodes.

5 Conclusion

The transition to the ABFR engine demonstrated that structural subproblem reduction vastly outperforms brute-force heuristic swapping. By replacing randomized guessing with mathematically guaranteed optimal sub-teams (Tree DP), the ABFR engine eliminated local optima traps. The algorithm broke a 91% success rate and crossed the 100% global relative yield threshold, meaning it now computes configurations that average higher than the pre-established mathematical baselines across the entire dataset.